SQL注入的本质

介绍

SQL注入是一种常见的网络攻击手段，攻击者通过在输入字段中插入恶意的SQL代码，来操纵数据库执行未授权的操作。这种攻击通常发生在Web应用程序对用户输入的数据没有进行严格的过滤和验证时。

原理

SQL注入的核心在于将用户输入的数据拼接到SQL语句中，并被当作代码执行。其产生需要满足两个条件：

1. **用户可控的参数**：前端传给后端的参数内容是用户可以控制的。
2. **参数带入数据库查询**：传入的参数拼接到SQL语句中，并带入数据库查询

JDBC介绍

JDBC (Java Database Connectivity) 是 Java 语言中用来规范客户端程序如何访问数据库的应用程序接口(API)，它提供了查询和更新数据库中数据的方法。

JDBC是Java企业应用开发中数据库访问的基础，大多数ORM框架(如Hibernate、MyBatis)底层都是基于JDBC实现的。

JDBC支持两种接口进行SQL语句执行，分别是

java.sql.Statement（不支持预编译）

java.sql.PreparedStatement（支持预编译）

那么我们就围绕这两个来说了

Statement

简单漏洞demo

```
@RestController
public class UserController {
    private final DataSource dataSource;

    public UserController(DataSource dataSource) {
        this.dataSource = dataSource;
    }
    @RequestMapping("/user")
    public String user(@RequestParam String id) {
        //直接拼接导致注入
        String sql = "select * from users where id =" + id;
        System.out.println(sql);

        try (Connection conn = dataSource.getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {

            if (rs.next()) {
```

柔狗安全内部培训资料

```
                return "用户名: " + rs.getString("username") + "<br>"
                        + "密码: " + rs.getString("password") + "<br>"
                        + "邮箱: " + rs.getString("email") + "<br>";
            } else {
                return "查询失败！";
            }

        } catch (SQLException e) {
            return "数据库错误: " + e.getMessage();
        }

    }
}
```

java中的字符串拼接方式

- +
- append()
- concat()
- join()
- format()
- add(

无注入情况分析

这个主要是涉及到数据流，我这里提到的无注入都是在漏洞demo的前提下的

- 数据类型转换
- 数据更改内容
- 存在严格过滤

我们一个个来说

数据类型转换

这个很好理解，比如我的网站存在个文章查询功能，通过传入id，带入数据库中查询对应文章内，那么我在开发这个功能的时候可能并不是考虑到了注入情况，而是考虑到传入非整型导致报错或者资源消耗的情况，于是我在参数介绍到拼接进数据库语句前进行了类型转换，或者在接收数据时就限定为整型，间接的预防了注入

```
@GetMapping("/search")
   public String search(@RequestParam int tid) {

       String sql = "select * from articles where id =" + tid;

       System.out.println(sql);

       try (Connection conn = dataSource.getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {
```

菜狗安全内部培训资料

```
            if (rs.next()) {
                return "查询文章id：" + rs.getString("id") + "<br>"
                        + "文章标题：" + rs.getString("title") + "<br>"
                        + "内容：" + rs.getString("content") + "<br>";
            } else {
                return "查询失败！";
            }
```

加密编码数据

这个也简单，就拿用户登录的功能点来说，开发者考虑到数据安全，对用户的密码进行了加密或者编码，然后传输到数据库中进行比对校验，那么假如说登录接口的密码存在直接拼接情况，那么我们传入的是个正常恶意语句例如：password=123456' or sleep(5) -- ,但是到后端经过处理(例如：md5加密)变成了a46a586ae3eaaf30aac628d46ed71697，那么也就失去了poc原本的功能，无法注入

```
    @PostMapping("/login")
    public String login(@RequestParam String username, @RequestParam String
password) throws IOException {
        String encryptedPassword = md5(password);

        String sql = "SELECT * FROM users WHERE username = '" + username +
                "' AND password = '" + encryptedPassword + "'";

        try (Connection conn = dataSource.getConnection();
             Statement stmt = conn.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {

            if (rs.next()) {
                return "登录成功！欢迎 " + rs.getString("username");
            } else {
                return "登录失败！";
            }
```

存在严格过滤

字符检测，字符替换都算是，这方面内容我相信师傅们已经非常熟悉了，看个简单的实战过滤规则把

```
 static final String[] INJECT_KEYWORD = new String[]{"'", "sitename", "net user",
"xp_cmdshell", "like'", "and", "exec", "execute", "insert", "create", "drop",
"table", "from", "grant", "use", "group_concat", "column_name",
"information_schema.columns", "table_schema", "union", "where", "select",
"delete", "update", "order", "by", "count", "chr", "mid", "master", "truncate",
"char", "declare", "or", ";", "-", "--", ",", "like", "%", "#", "*", "+"};
    static final String[] KEYWORD_FULL_STR = new String[]{"′", "sitename",
"net user", "xp_cmdshell", "like′", "and", "exec", "execute", "insert", "create", "drop", "table", "from", "grant",
"use", "group_concat", "column_name", "information_schema.columns", "table_schema", "union", "where", "select",
"delete", "update", "order", "by", "count", "chr", "mid", "master", "truncate", "char", "declare", "or", ";", "—",
"——", ",", "like", "%", "#", "*", "+"};
```

菜狗安全内部培训资料

那么在遇到过滤十分严格的情况下，我们要关注的就是没有进行过滤的注入点了，关注过滤规则是否全局启用，例如过滤器或拦截器，还是封装方法在查询点提前使用，存不存在遗漏，这个是我们审计的关注点

审计实战案例

MRCMS

Wangmarket

网站管理后台

登陆地址： http://127.0.0.1:8080/login.do

登陆账号： **wangzhan**

登陆密码： **wangzhan**

代理后台

登陆地址： http://127.0.0.1:8080/login.do

登陆账号： **agency**

登陆密码： **agency**

总管理后台

登陆地址： http://127.0.0.1:8080/login.do

登陆账号： **admin**

登陆密码： **admin**

student-manage

PreparedStatement

`PreparedStatement` 是 Java JDBC API 中的一个核心接口，继承自 `Statement` 接口，用于执行预编译的 SQL 语句。它是防止 SQL 注入攻击的首选方式，同时也能提高数据库操作性能。

核心特性

1. **预编译机制**：
    - SQL 语句被预先编译并存储在数据库中
    - 后续执行只需传递参数值，无需重新编译
2. **参数化查询**：
    - 使用占位符 (`?`) 代替直接拼接值
    - 通过 `setXxx()` 方法安全地设置参数
3. **自动类型转换**：
    - 自动处理 Java 类型与 SQL 类型的转换

不用太在乎这个，除了这个还有非常多基于原生JDBC的数据库操作API

素雅安全内部培训资料

DataSource接口

```
DataSource ds = // 获取DataSource实例
try (Connection conn = ds.getConnection();
     PreparedStatement ps = conn.prepareStatement("INSERT...")) {
    // 设置参数并执行
}
```

第三方库

HikariCP (连接池)

```
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:mysql://localhost/test");
config.setUsername("user");
config.setPassword("password");

try (HikariDataSource ds = new HikariDataSource(config);
     Connection conn = ds.getConnection()) {
    // 使用连接
}
```

Apache Commons DbUtils

```
QueryRunner run = new QueryRunner(dataSource);
// 查询
List<Map<String, Object>> results = run.query("SELECT * FROM users",
    new MapListHandler());
// 更新
run.update("UPDATE users SET name=? WHERE id=?", "John", 1);
```

jOOQ (类型安全的SQL构建)

```
DSLContext create = DSL.using(connection, SQLDialect.MYSQL);
Result<Record> result = create.select().from(AUTHOR).fetch();
```

MyBatis (SQL映射框架)

```
SqlSession session = sqlSessionFactory.openSession();
try {
    UserMapper mapper = session.getMapper(UserMapper.class);
    User user = mapper.selectUser(1);
} finally {
    session.close();
}
```

柔韵安全内部培训资料

JPA (Java Persistence API)

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("myPU");
EntityManager em = emf.createEntityManager();
User user = em.find(User.class, 1);
```

Spring Data JPA (基于JPA的Repository抽象)

```
public interface UserRepository extends JpaRepository<User, Long> {
    User findByName(String name);
}
```

R2DBC (响应式JDBC)

```
ConnectionFactory connectionFactory =
ConnectionFactories.get("r2dbc:mysql://user:password@localhost/test");
Mono.from(connectionFactory.create())
    .flatMapMany(connection ->
        connection.createStatement("SELECT * FROM users").execute())
    .flatMap(result -> result.map((row, meta) -> row.get("name", String.class)))
    .subscribe(System.out::println);
```

菜狗安全内部培训资料